Open-Source French TTS Web Integration

https://gemini.google.com/app/84596315af75f6a5

User prompt: could you reserach all projects similar to this, https://github.com/YASSER-27/Supertonic-Voice I have built a web app and those voices from supertonic, most of them are top notch, I have tried eleven labs but I would like to keep adding good free tier local computing voices, could you grab the github projects that provide TTS for french?

Response: `YASSER-27/Supertonic-Voice` is a desktop and Android wrapper built around **Supertonic** (`supertone-oss-archive/supertonic`), an open-weight ONNX text-to-speech engine that runs locally on CPU or directly inside the browser via `onnxruntime-web`. While Supertonic 3 expanded its support to 31 languages (including French), its preset style embeddings (`F1`–`F5`, `M1`–`M5`) are fixed, so expanding your French voice roster requires bringing in additional open-source TTS engines.

Because you are integrating these into a web app, the best GitHub repositories split into two architectural paths: **In-Browser (WASM / WebGPU)** engines that run directly on the client with zero server overhead, and **Local Server / API** engines that deliver ElevenLabs-grade French prosody and zero-shot cloning from a local backend or pre-generation script.

1. Direct Supertonic Ecosystem & In-Browser (WASM / ONNX) Engines

If you want to keep your architecture purely client-side in JavaScript (like `onnxruntime-web`) or lightweight CPU-only without spinning up a GPU server, these are the direct equivalents to Supertonic:

- **Upstream Supertonic (`supertone-oss-archive/supertonic`) & Rust Server (`go-restream/supertts`):**
  - **What it is:** The official upstream repository for the models used in `Supertonic-Voice`, alongside a high-performance Rust wrapper (`supertts`) that exposes an OpenAI-compatible `/v1/audio/speech` endpoint with voice-style pooling.
  - **French Support:** Native French support in **Supertonic 3** (~4.89% Word Error Rate on multilingual benchmarks). If your current setup is still on Supertonic 2 weights, upgrading to the `supertonic-3-fp16-v2` ONNX bundle immediately improves French pronunciation and reduces word-skipping.
- **Piper TTS Web (`Mintplex-Labs/piper-tts-web` / `diffusionstudio/vits-web`) & Upstream (`OHF-Voice/piper1-gpl`):**
  - **What it is:** A pure JavaScript/WASM library that runs Piper's ONNX VITS models directly in the browser and caches the voice weights inside the browser's Origin Private File System (OPFS). (Note: The original `rhasspy/piper` repo moved to `OHF-Voice/piper1-gpl` under the Open Home Foundation).
  - **French Voices:** Gives you immediate access to **6 distinct French voice models** (`fr_FR`) hosted under `rhasspy/piper-voices`: `upmc` (very natural male/female academic dataset), `siwis` (crisp female voice), `tom`, `gilles`, and multi-speaker `mls` models.
  - **Why use it:** Ultra-fast, zero backend cost, MIT-licensed weights, and drops straight into a vanilla JS or React frontend with 5 lines of code.
- **Sherpa-ONNX (`k2-fsa/sherpa-onnx`):**
  - **What it is:** The Swiss Army knife of local ONNX speech synthesis. It supports WebAssembly (WASM), Node.js addons, Python, C++, and mobile targets, wrapping VITS, Piper, Matcha-TTS, and Kokoro models into a unified runtime.
  - **French Support:** Provides pre-packaged `.tar.bz2` ONNX bundles for French VITS and Piper models ready for offline WASM or local server deployment.
- **Kokoro TTS (`hexgrad/kokoro`):**
  - **What it is:** A viral 82M-parameter StyleTTS2-based model (Apache 2.0) that punches way above its weight class—matching massive models in blind tests while running real-time on a standard CPU.
  - **French Support & JS Caveat:** Includes a fantastic French female voice (`ff_siwis`). **Important developer note:** While the Python package handles French phonemization smoothly, the npm package (`kokoro-js`) defaults to English phoneme rules unless you pass pre-phonemized tokens or run the Python backend.

2. ElevenLabs-Grade Local Engines (High-Expressiveness & Voice Cloning)

If you want voices that truly rival ElevenLabs in French conversational flow, liaison handling, and emotional inflection—or if you want to generate unlimited custom French voices from 5-second audio clips—these open-source engines can be run locally via Docker/Python and called from your web app via REST API:

- **CosyVoice2-EU (`hi-paris/CosyVoice2-EU`):**
  - **What it is:** Built by the Hi! PARIS research center specifically to fix the Asian/English accent bias in standard open-source TTS models. It is an Apache 2.0 bilingual (French & German) adaptation of `FunAudioLLM/CosyVoice2` (0.5B parameters).

- **Why it stands out for French:** Because it was explicitly fine-tuned for European French phonetics and zero-shot voice cloning, its French cadence, nasal vowels, and intonation are arguably the most authentic among lightweight (0.5B) open models. Installable via `pip install cosyvoice2-eu` with streaming support.

- **Chatterbox TTS (`resemble-ai/chatterbox` & `travisvn/chatterbox-tts-api`):**
  - **What it is:** Resemble AI's MIT-licensed 350M/500M multilingual model supporting 23 languages, including French.
  - **Why it stands out:** Beats ElevenLabs in blind A/B preference tests, allows paralinguistic tags (like `[chuckle]` or `[sigh]`), and offers an `exaggeration` and `cfg_weight` dial to tune pacing. Pairing it with `travisvn/chatterbox-tts-api` gives your web app a local OpenAI-compatible endpoint (`/v1/audio/speech`) where you can drop in any 10-second native French `.wav` file to create a permanent new voice profile.

- **Qwen3-TTS (`QwenLM/Qwen3-TTS`):**
  - **What it is:** Alibaba's Apache 2.0 TTS suite (0.6B and 1.7B models) with native French support.
  - **Why it stands out:** In the benchmark published by the Supertonic team, `Qwen3-TTS` achieved a **3.82% Word Error Rate in French**—beating Supertonic 3 (4.89%). Even better, its `VoiceDesign` model lets you *invent* new French voices using text prompts (e.g., *"A warm, middle-aged French male professor speaking slowly and clearly"*), which you can then save and reuse across your app.

- **Ming-3.0-TTS (`inclusionAI/Ming-3.0-tts`):**
  - **What it is:** Released with a compact **0.5B dense** model alongside a larger MoE variant, Ming-3.0-tts ranks #1 among open-weight models on pronunciation robustness and controlled voice benchmarks, supporting multilingual synthesis, voice design, and LoRA fine-tuning.

- **MeloTTS (`myshell-ai/MeloTTS`):**
  - **What it is:** An MIT-licensed library optimized specifically for fast, real-time CPU inference without requiring a GPU, featuring built-in French support. Great middle ground if you want a Python backend without heavy VRAM requirements.

- **Fish Speech S2 Pro (`fishaudio/fish-speech`) & OmniVoice (`k2-fsa` / `VoiceStudio`):**
  - **What they are:** Ultra-expressive zero-shot models with top-tier French quality (Fish Speech was trained on >20,000 hours of French audio; OmniVoice covers 600+ languages at 0.6B parameters).
  - **License Caveat:** Both use research/non-commercial licenses for their latest weights (`CC BY-NC` for OmniVoice; `Fish Audio Research/Community License` for Fish Speech). They are phenomenal for personal projects, local study tools, or prototyping, but stick to **Supertonic, Piper, Kokoro, CosyVoice2-EU, Chatterbox, or Qwen3-TTS** if you need unrestricted commercial rights.

3. Side-by-Side Comparison of French Local TTS Repositories

GitHub Repository	Model Size	Runs In-Browser (JS/WASM)?	Hardware Needed	French Voice Variety	License
`supertone-oss-archive/supertonic`	~200 MB (FP16)	**Yes** (`onnxruntime-web`)	CPU / Browser	10 preset styles (`F1`–`F5`, `M1`–`M5`)	MIT
`Mintplex-Labs/piper-tts-web`	15–65 MB / voice	**Yes** (WASM + OPFS cache)	CPU / Browser	6+ native `fr_FR` voices (`siwis`, `upmc`, `tom`, `gilles`)	MIT
`hexgrad/kokoro`	82M params	Partial (needs FR phonemizer)	CPU (90x+ real-time)	1 native voice (`ff_siwis`) + voice mixing	Apache 2.0
`myshell-ai/MeloTTS`	~150 MB	No (Python API / Docker)	CPU-optimized	Built-in native French speaker	MIT
`hi-paris/CosyVoice2-EU`	0.5B params	No (Python CLI / FastAPI)	GPU (4GB+ VRAM) or fast CPU	**Infinite** (Zero-shot cloning optimized for EU French)	Apache 2.0
`resemble-ai/chatterbox`	350M / 500M	No (OpenAI-compatible API)	GPU (4GB+ VRAM) / CPU (Nano)	**Infinite** (5s zero-shot cloning + emotion control)	MIT
`QwenLM/Qwen3-TTS`	0.6B & 1.7B	No (Python / vLLM server)	GPU (6GB–16GB VRAM)	**Infinite** (9 presets + text-prompted Voice Design + 3s clone)	Apache 2.0
`inclusionAI/Ming-3.0-tts`	0.5B Dense / 1.3B MoE	No (Python / vLLM-Omni)	GPU (4GB+ VRAM)	**Infinite** (Zero-shot cloning + natural language control)	Apache 2.0

GitHub Repository	Model Size	Runs In-Browser (JS/WASM)?	Hardware Needed	French Voice Variety	License
`fishaudio/fish-speech`	4B (S2-Pro)	No (API Server / Docker)	GPU (8GB+ VRAM)	**Infinite** (10s clone + inline tags like `[whisper]`)	Research / NC

Architectural Recommendation for Your Web App

1. **For Instant Client-Side Expansion (Zero Server Cost):** Add **`Mintplex-Labs/piper-tts-web`** right alongside your Supertonic implementation. Because both use ONNX/WASM in the browser, you can let the user toggle between Supertonic's `F1`–`M5` voices and Piper's `fr_FR-upmc-medium` and `fr_FR-siwis-medium` voices seamlessly, caching the models locally in the browser.
2. **For Pre-Generating Audio Assets or Running a Local Backend:** Spin up **`hi-paris/CosyVoice2-EU`** or **`resemble-ai/chatterbox`** (via `travisvn/chatterbox-tts-api`). You can grab 10-second clips of clear native French speakers (for instance, from CommonVoice or Librivox French audiobooks) and generate as many distinct, hyper-realistic French voices as your app needs without ever hitting a paid API limit.

Sources

1. GitHub - supertone-oss-archive/supertonic: Lightning-Fast, On
2. rhasspy/piper: A fast, local neural text to speech system - GitHub
3. GitHub - k2-fsa/sherpa-onnx: Speech-to-text, text-to-speech, speaker
4. Kokoro TTS in Node.js - Micdrop
5. CosyVoice2-EU - hi-paris - GitHub
6. chatterbox-tts-api/docs/MULTILINGUAL.md at main - GitHub
7. inclusionAI/Ming-3.0-tts - GitHub
8. 6 Popular Open-Source Text-to-Speech Models in 2026 - Hyperstack

---

User prompt: I also have a very rudemantry, made by vibe coding Speech to Text to assess shadowing in french, what are my options to make it more robust but not to do it with the paid AI engines, could you research

Response: Most "vibe-coded" browser Speech-to-Text setups rely on the browser's built-in `webkitSpeechRecognition` (Web Speech API) or a basic unconfigured Whisper call. While that works for general dictation, it struggles with **French shadowing assessment** for two reasons:

1. **Silent Homophones:** In French, *il parle*, *ils parlent*, and *tu parles* sound identical in isolation. A standard STT engine guesses based on context—or worse, auto-corrects a learner's mispronunciation into valid French text so you can't tell if they actually pronounced the liaison or nasal vowel properly.
2. **Latency & Padding:** Standard Whisper pads every audio clip to 30 seconds (`30s` mel-spectrogram chunks), which makes short 3-second shadowing phrases sluggish unless you use an engine that processes audio at its true length.

Moving to a robust, 100% free, local-compute architecture for French shadowing involves three open-source upgrade paths—ranging from pure in-browser WebGPU/WASM to acoustic phonetic scoring.

1. Pure In-Browser Engines (WASM / WebGPU — Zero Server Cost)

Running the STT engine directly in the user's browser alongside your Supertonic/Piper TTS voices avoids backend costs using these top open-source stacks:

- **Transformers.js v3 (`huggingface/transformers.js`) + French Distil-Whisper:**
  - **Why it beats standard Whisper:** Instead of loading the generic multilingual `whisper-tiny` or `whisper-base` (which have high error rates in French), load **`bofenghuang/whisper-large-v3-french-distil-dec4`** (or `onnx-community/whisper-small` quantized to `q8`/`fp16`) via WebGPU.
  - **Shadowing Advantage:** Pass `{ return_timestamps: "word" }` to the pipeline. This gives you exact **start and end timestamps for every French word**, allowing your UI to highlight words in real time and measure whether the user matched the native TTS speaker's cadence and rhythm.
- **Moonshine JS (`usefulsensors/moonshine` + `Cornebidouil/moonshine-tiny-fr`):**
  - **Why it stands out:** Unlike Whisper, which pads every 2-second shadowing recording to 30 seconds before running inference, **Moonshine** scales compute strictly to the actual length of the audio.
  - **French Model:** `Cornebidouil/moonshine-tiny-fr` is a 27M-parameter MIT-licensed model fine-tuned specifically on French Multilingual LibriSpeech. It runs ~9x faster than real-time on CPU/WASM and delivers ~100ms latency from the end of speech to final transcript.
- **Sherpa-ONNX WebAssembly (`k2-fsa/sherpa-onnx`) + NVIDIA Parakeet TDT:**

- **Why it stands out:** `sherpa-onnx` compiles to WASM and Node.js addons and runs **NVIDIA's `parakeet-tdt-0.6b-v3`** (600M parameters, CC-BY-4.0, supporting 25 European languages including French).
- **Shadowing Advantage:** Parakeet uses a Token-and-Duration Transducer (TDT) decoder rather than Whisper's autoregressive loop, making it 5x to 10x faster on CPU while outputting word-level timestamps and never hallucinating repeated phrases on silence.

- **Vosk Browser (`ccoreilly/vosk-browser` + `alphacep/vosk-api`):**
  - **Why it stands out:** Ultra-lightweight (`vosk-model-small-fr-0.22` is only ~40MB) and runs in a Web Worker.
  - **The Killer Feature for Shadowing (Constrained Grammar):** Because Vosk is Kaldi-based, it allows you to pass a **constrained vocabulary/grammar array** containing the target sentence plus common phonetic mistakes. It also outputs a **per-word acoustic confidence score (`0.0` to `1.0`)**, which tells you not just *what* word was recognized, but *how clearly* each French word was pronounced.

2. True Pronunciation & Shadowing Assessment (Forced Alignment & Phonemes)

In a shadowing app, you **already know the exact French sentence** the user is supposed to be repeating. Using a blind transcription model means fighting the language model's tendency to "fix" bad pronunciation.

Professional language apps (like Duolingo or Elsa Speak) use **CTC Forced Alignment** and **Goodness of Pronunciation (GoP)** instead of plain STT:

- **Wav2Vec2-CTC French (`bofenghuang/asr-wav2vec2-ctc-french` or `facebook/wav2vec2-large-xlsr-53-french`):**
  - **How it works:** Unlike Whisper (which uses a heavy language decoder that guesses grammar), Wav2Vec2 is a pure **acoustic model** trained with Connectionist Temporal Classification (CTC). It maps 20ms audio frames directly to French characters/phonemes without "autocorrecting" grammar.
  - **Why it's better for shadowing:** If the user forgets to pronounce a liaison (e.g., *les amis* without the `/z/` sound) or mispronounces *u* (`/y/`) as *ou* (`/u/`), Wav2Vec2's raw probability matrix drops immediately on those specific letters, giving you an exact per-syllable pronunciation score.
  - **In-Browser Support:** Works natively in `transformers.js` using the `automatic-speech-recognition` pipeline (`Xenova/wav2vec2-large-xlsr-53-french`).
- **CTC Forced Aligner (`MahmoudAshraf97/ctc-forced-aligner`):**
  - **How it works:** Takes two inputs: the **User's Audio** + the **Target French Text**. It forces the acoustic model to align every single word and character of the target French sentence to the user's audio recording and returns a JSON object with exact start/end timestamps and a **confidence probability score per word/character**.
- **WhisperX (`m-bain/whisperX`):**
  - **How it works:** If you run a local Python backend, WhisperX combines `faster-whisper` (for ultra-fast transcription) with a French Wav2Vec2 phoneme alignment pass. It outputs millisecond-accurate word and character timings along with confidence scores.

3. Architecture Upgrade Tricks for Your Current Web App

Even before swapping out your entire engine, you can immediately make your French shadowing assessment far more accurate with four open-source engineering upgrades:

1. **Add Client-Side Voice Activity Detection (`ricky0123/vad` / Silero VAD):**
   - Whisper and browser STT models hallucinate wildly (often outputting *"Sous-titres réalisés par la communauté d'Amara.org"* or *"Merci d'avoir regardé"*) when fed leading or trailing silence. Wrapping your mic input in `@ricky0123/vad-web` (which runs a 2MB Silero VAD ONNX model in the browser) trims silence automatically and triggers transcription the exact millisecond the user finishes shadowing.
2. **Compare Phonemes (IPA), Not Spelling (`espeak-ng` / `phonemizer`):**
   - Never grade a French learner using raw string distance on French spelling—otherwise, if Whisper transcribes *"Il mange"* instead of *"Ils mangent"*, your app marks the user wrong even though both are pronounced `/il mɑ̃ʒ/`.
   - **The Fix:** Convert both the target sentence and the STT output into **French Phonemes (IPA)** or use a French phonetic normalizer (stripping silent endings like `-ent`, `-es`, `-s` when not followed by a vowel) before running Levenshtein distance.
3. **Use `initial_prompt` Biasing in Whisper:**
   - If you use Whisper (`transformers.js` or `whisper.cpp`), pass the target French vocabulary or topic in the prompt parameter. This prevents Whisper from guessing obscure homophones while still failing if the user's acoustic pronunciation is genuinely off.

4. **Compare Rhythm & Speech Rate (Prosody Matching):**
    - Since you generate the reference audio locally via Supertonic/Piper, you know the exact duration of the native TTS clip (e.g., `2.4 seconds`). By comparing the user's VAD-trimmed speech duration and word timestamps against the TTS audio, you can score their **prosody and fluidity** alongside their word accuracy.

4. Comparison of Open-Source French STT Options for Shadowing

Project / Model	Runs In-Browser?	Model Size	Best Feature for French Shadowing	License
`transformers.js` + `whisper-small` / `distil-fr`	**Yes** (WebGPU / WASM)	150–400 MB (q8)	Word-level timestamps (`return_timestamps: 'word'`)	Apache 2.0 / MIT
`Xenova/wav2vec2-large-xlsr-53-french`	**Yes** (`transformers.js`)	~300 MB (q8)	Pure acoustic CTC output—doesn't "autocorrect" bad pronunciation	Apache 2.0
`Cornebidouil/moonshine-tiny-fr`	**Yes** (`@moonshine-ai/moonshine-wasm`)	~27M params (~30 MB)	No 30-second padding; instant (~100ms) on short shadowing phrases	MIT
`ccoreilly/vosk-browser` (`fr-0.22`)	**Yes** (WASM WebWorker)	~42 MB	Constrained grammar + per-word acoustic confidence scores	Apache 2.0
`k2-fsa/sherpa-onnx` + `parakeet-tdt-0.6b-v3`	**Yes** (WASM / Node.js)	~600 MB	5–10x faster than Whisper, zero silence hallucinations	Apache 2.0 / CC-BY-4.0
`MahmoudAshraf97/ctc-forced-aligner`	No (Python Backend)	~300 MB–1 GB	Aligns known French text to user audio & scores every character	MIT

Sources

1. Local Speech to Text in Node: Whisper and Its Alternatives - Micdrop
2. Free Speech to Text Online | Whisper AI Transcription in Browser 2026
3. README.md · Cornebidouil/moonshine-tiny-fr at main - Hugging Face
4. Speech to Text | Moonshine Voice
5. The NVIDIA Parakeet model - Whisper by Remskill

User prompt: I have a hostinger hosting, would that limit any of the architecture decitions? I could maybe buy a VPS or something but the app right now doesn't generate revenue

Response: Keeping your app on standard Hostinger shared or cloud web hosting will **not** hold you back—and you should **not** buy a VPS while the app is pre-revenue.

Even if you upgraded to an entry-level Hostinger VPS (such as KVM 1 or KVM 2 at $4.99 to $7.64/month), you only get 1 to 2 CPU cores and 4 to 8 GB of RAM with **no dedicated GPU**. Running heavy PyTorch TTS or STT models (like `CosyVoice2-EU`, `Qwen3-TTS`, or `WhisperX`) live on a cheap CPU-only VPS would result in slow, multi-second queues whenever two users practice at the same time.

Instead, you can run a high-performance architecture at **$0 extra cost** on your existing Hostinger plan by combining client-side WebAssembly/WebGPU inference with pre-generated static audio.

1. What Works on Hostinger Shared Hosting vs. What Is Blocked

Hostinger's shared and cloud plans run on LiteSpeed web servers designed to serve static files and PHP, without root access, custom open ports, or persistent background Python daemons. Because **In-Browser (WASM / WebGPU)** models run entirely on your **user's CPU/GPU** rather than Hostinger's server, all client-side engines work on your current plan with zero server load:

Component	Compatible with Hostinger Shared Hosting ($0 Extra)	Blocked on Shared Hosting (Requires GPU Server / VPS)
French TTS	• **Supertonic** (`onnxruntime-web`) • **Piper TTS Web** (`@mintplex-labs/piper-tts-web`)	• Live **CosyVoice2-EU** Python API • Live **Chatterbox TTS** FastAPI server

Component	Compatible with Hostinger Shared Hosting ($0 Extra)	Blocked on Shared Hosting (Requires GPU Server / VPS)
	• **Sherpa-ONNX WASM** (VITS / Matcha / Kokoro)	• Live **Qwen3-TTS** / **Fish Speech** servers
French STT & Shadowing	• **Transformers.js v3** (`whisper-small` / `distil-fr`) • **Wav2Vec2-CTC French** (`Xenova/wav2vec2-large-xlsr-53-french`) • **Moonshine JS** (`Cornebidouil/moonshine-tiny-fr`) • **Vosk Browser** (`vosk-model-small-fr-0.22`) • **Silero VAD** (`@ricky0123/vad-web`)	• Live **WhisperX** Python backend • Live **MahmoudAshraf97/ctc-forced-aligner** server • Heavy server-side PyTorch phoneme aligners

2. Three $0 Architecture Strategies to Get "VPS Quality" on Hostinger

A. Pre-Generate "ElevenLabs-Grade" Audio Locally (The Hybrid Trick)

In a French shadowing app, 90% of your curriculum (dialogues, drills, and target sentences) is fixed text. You do not need a live server to synthesize the same French sentence 500 times:

1. Run **`CosyVoice2-EU`**, **`Chatterbox`**, or **`Qwen3-TTS`** locally on your own computer to batch-generate `.mp3` or `.opus` audio files (plus a `.json` file of word timestamps) for your structured lessons.
2. Upload those lightweight static `.mp3` and `.json` files to your Hostinger hosting (or Cloudflare R2's free 10 GB tier).
3. Use in-browser **Supertonic** or **Piper TTS Web** strictly for dynamic, user-typed custom practice sentences.

B. Let Hugging Face CDN Serve the Model Weights (Save Your Hostinger Quota)

Hostinger shared plans enforce storage caps (10 GB to 50 GB depending on your plan version) and fair-use policies that prohibit using your web hosting as a file-distribution mirror.

- **Do not** upload 200 MB to 500 MB `.onnx` or `.wasm` model files into your Hostinger `public_html` folder.
- By default, `transformers.js` and `piper-tts-web` fetch model weights directly from **Hugging Face's free global CDN** and cache them permanently inside the user's browser via the **Origin Private File System (OPFS)** or **Cache API**. Your Hostinger server only serves a few kilobytes of HTML, CSS, and vanilla JS.

C. Use Hugging Face Spaces (Free Tier) If You Need a Python Backend

If you ever want to run a Python-only tool—such as `ctc-forced-aligner` for character-level French phonetic grading—don't rent a Hostinger VPS. Deploy a free Docker container on **Hugging Face Spaces (Free CPU Basic gives you 2 vCPUs and 16 GB RAM for $0)** and call its API endpoint directly from your Hostinger frontend.

3. Critical Hostinger `.htaccess` Configuration for WebAssembly Speed

By default, modern browsers disable multi-threaded WebAssembly (`SharedArrayBuffer`) unless your server sends specific **Cross-Origin Isolation** HTTP headers. Without these headers on Hostinger, `onnxruntime-web` (Supertonic, Piper, and Transformers.js) silently drops back to single-threaded CPU execution, making TTS and STT **2x to 4x slower**.

Because Hostinger runs LiteSpeed with `.htaccess` support, add the following rules to your `public_html/.htaccess` file to unlock full multi-threaded WASM performance while still allowing your app to download models from external CDNs like Hugging Face:

```
<IfModule mod_headers.c>
  # Enable SharedArrayBuffer for multi-threaded ONNX / WASM inference
  Header set Cross-Origin-Opener-Policy "same-origin"
  # Use "credentialless" instead of "require-corp" so external CDN fetches (Hugging Face) aren't blocked
  Header set Cross-Origin-Embedder-Policy "credentialless"
</IfModule>
```

```
<IfModule mod_mime.c>
  # Ensure LiteSpeed serves WebAssembly and ONNX files with proper MIME types
  AddType application/wasm .wasm
  AddType application/octet-stream .onnx
</IfModule>
```

Verification: Open your live Hostinger site in Chrome DevTools, go to the **Console**, and type `window.crossOriginIsolated`. If the `.htaccess` headers are working, it will return `true` instead of `false`, confirming multi-threaded WASM is active.

Sources

1. Hostinger Python Hosting Review: The Right Choice for 2026?
2. How to enable sharedarraybuffer on website - TechKBlog

User prompt: oh I menat for the TTS shadowing stuff, for audio I generate that offline

Response: Generating the French TTS audio offline on your own machine removes all server constraints from voice synthesis and makes the **live shadowing assessment on Hostinger** much easier to engineer.

Because you create the reference audio ahead of time on your computer, you can shift 50% of the heavy shadowing assessment work into your **offline audio generation script**, leaving only a lightweight evaluation step for the live web app on Hostinger.

1. The "Offline Pre-Alignment" Advantage

In a live shadowing app, you normally have to figure out *what* the reference audio says and *when* each word occurs. Because you generate the French TTS files offline on your local computer, you can run a **Forced Aligner** right after generating each `.mp3` file to export a tiny companion `.json` file alongside the audio:

1. **Generate the French `.mp3` offline** using any local Python model on your machine (`Supertonic 3`, `CosyVoice2-EU`, `Chatterbox`, or `Kokoro`).
2. **Run `WhisperX` or `stable-ts` (`force-align-wordstamps`) offline** on that `.mp3` plus the French script. Because forced alignment already knows the exact text, it maps every word and phoneme to its exact millisecond start and end time without drifting.
3. **Upload both `phrase_01.mp3` and `phrase_01.json` to Hostinger.**

Your static `.json` file on Hostinger looks like this:

```
{
  "text": "Les amis sont arrivés tôt.",
  "ipa": "lez‿ami sɔ̃t‿aʁive to",
  "duration_ms": 2150,
  "words": [
    { "word": "Les", "start": 0.05, "end": 0.22, "ipa": "lez" },
    { "word": "amis", "start": 0.22, "end": 0.68, "ipa": "ami" },
    { "word": "sont", "start": 0.68, "end": 0.95, "ipa": "sɔ̃t" },
    { "word": "arrivés", "start": 0.95, "end": 1.62, "ipa": "aʁive" },
    { "word": "tôt", "start": 1.62, "end": 2.10, "ipa": "to" }
  ]
}
```

Having this pre-computed `.json` on Hostinger gives your frontend instant karaoke-style word highlighting during playback and a "golden template" to grade the learner against.

2. How Hostinger Affects the Live Shadowing Assessment (STT)

When the user speaks into their microphone to shadow the phrase, **Hostinger shared hosting cannot run Python (`faster-whisper`, `WhisperX`, or PyTorch) on the server** to grade their recording.

However, you still **do not need to buy a VPS**. You have three $0 ways to grade the user's microphone recording while hosted on Hostinger:

Option A: 100% In-Browser WASM Assessment (Zero Backend)

Your Hostinger site serves the static HTML/JS, and the **user's browser** runs a small quantized French acoustic model inside a Web Worker:

- **How it works:** Load `Cornebidouil/moonshine-tiny-fr` (~30 MB), `vosk-browser` (`vosk-model-small-fr-0.22`) (~42 MB), or `Xenova/wav2vec2-large-xlsr-53-french` via `transformers.js` in the browser.
- **How it grades:** When the user finishes speaking, the in-browser model returns the recognized French words/phonemes, per-word confidence probabilities, and word timestamps. Your vanilla JS compares the user's output against your pre-computed

`phrase_01.json`.

- **Hostinger requirement:** Only requires the `.htaccess` `SharedArrayBuffer` headers so WebAssembly runs multi-threaded in the user's browser.

Option B: Direct Audio-to-Audio Acoustic Matching in Vanilla JS (~50 KB, Zero AI Download)

Because you already have the clean offline TTS `.mp3` loaded in the browser, you can grade the user's **rhythm, intonation, and vowel quality** by comparing their microphone audio buffer directly against the TTS audio buffer using the browser's native **Web Audio API**:

- **How it works:** Use **`Meyda`** (a lightweight JavaScript audio feature extractor) to extract **MFCCs (Mel-Frequency Cepstral Coefficients)** and pitch contours from both the offline TTS audio and the user's mic recording, then align them using **Dynamic Time Warping (`dtw`)** in pure JavaScript.
- **Why it's great for shadowing:** Even if a basic browser STT engine auto-corrects a learner's spelling, MFCC + DTW acoustic comparison measures how closely their pronunciation and cadence physically matched the native TTS recording—with zero heavy model downloads and zero server CPU usage.

Option C: Hostinger Frontend + Free Hugging Face Space Worker (Server-Side Python for $0)

If you don't want your users' browsers downloading a 30 MB to 150 MB WASM model—especially on older mobile phones—offload the Python audio grading to **Hugging Face Spaces (Free CPU Basic tier: 2 vCPUs, 16 GB RAM, $0/month)**:

- **How it works:** Your main web app stays on Hostinger. When a user records a 3-second shadowing clip, your JS sends the `.wav` blob and the target French text to a small FastAPI container on your free Hugging Face Space running **`MahmoudAshraf97/ctc-forced-aligner`** or **`faster-whisper`**.
- **How it grades:** The Python worker force-aligns the user's voice to the known French sentence and returns a JSON score for every single word and liaison in ~300ms.

3. Comparison of $0 Shadowing Assessment Setups for Hostinger

Architecture	Client Download Size	Catches Silent Homophones / Liaisons?	Server Cost	Best For
1. Offline JSON + In-Browser `Wav2Vec2-CTC` / `Moonshine`	30 MB – 150 MB (cached once)	**Yes** (CTC scores raw acoustics without grammar autocorrect)	**$0** (Runs on user's device)	Desktop & modern mobile browsers with offline caching
2. Web Speech API + JS Acoustic DTW (`Meyda`)	Less than 50 KB	**Yes** (Compares user audio directly to your offline TTS `.mp3`)	**$0** (Runs on user's device)	Instant page loads with zero model download wait
3. Hostinger UI + Free Hugging Face Space API	0 MB	**Yes** (Full Python `ctc-forced-aligner` / `WhisperX` grading)	**$0** (Free 16GB RAM HF Space)	Low-end mobile devices where WASM might lag

Sources

1. Word timestamps and forced alignment in speech-to-text - Soniox

---